QuantSolve: A Self-Verifying, Self-Healing Code-Execution Agent for Quantitative Problems Using ReAct and Reflexion

Deon Menezes
Dept. of Artificial Intelligence & Data Science
K J Somaiya School of Engineering
Somaiya Vidyavihar University, Mumbai, India
Roll No. 16014223030

Aishwarya Gawali
Dept. of Artificial Intelligence & Data Science
K J Somaiya School of Engineering
Somaiya Vidyavihar University, Mumbai, India
Roll No. 16014223007

GitHub repository (code, data and web application): https://github.com/DeonMenezes-11/QuantSolve

***Abstract*—Large language models (LLMs) write plausible code quickly, but on multi-step quantitative problems a single sample often contains small slips (an off-by-one loop bound, a flipped sign, a misread rule) that produce a confident but wrong number. We present QuantSolve, an agent that answers such problems through program-aided reasoning inside a self-correction loop. The model writes a complete Python program whose answer must be returned in a `RESULT` dictionary and verified by at least two `assert` self-checks that compare quantities computed in different ways. The program runs in a layered sandbox built around a restricted `exec()` in a separate process. Every failure (blocked code, syntax and runtime errors, failed self-checks, timeouts, contract violations) is returned to the model as a cleaned observation, and a Reflexion-style verbal reflection is stored in memory for later attempts. On 8 problems with hand-written reference answers, run 2 times per arm with `gpt-4.1-mini`, the loop raised accuracy from 69% (one-shot) to 81% at 1.19 attempts per run on average, and recovered 5 of 6 runs with a deliberately injected crash. Injected logic bugs were harder: the model's own self-checks caught 2 of the 9 attributable bugs. The sandbox contained all 17 escape and resource-exhaustion attacks we tried. We also describe a web application that streams every step of the loop to the browser in real time.**

***Index Terms*—large language models, program-aided reasoning, self-correction, ReAct, Reflexion, self-verification, code sandboxing**

I. Introduction

Numerical word problems (loan amortisation, step-up investments, income tax with rebates and marginal relief, projectile motion with drag, exact probabilities) are a natural fit for program-aided reasoning: the model describes the computation in code and the interpreter performs the arithmetic [3], [4]. Code, however, introduces its own failure modes. A program can crash, never terminate, or, worst of all, run to completion and print a wrong number with no visible error.

The lab problem behind this work asks for a *self-correction "code execution and auto-heal" loop*: take a complex task, generate Python, execute it safely, capture the traceback on failure and iteratively fix the code. We follow the ReAct pattern of interleaving reasoning, actions and observations [1] and the Reflexion idea of improving across trials through verbal self-reflection [2]. Two design choices make the loop useful beyond crash repair. First, every program must verify its own answer with assertions, so that some logic errors surface as observable failures. Second, generated code runs behind several independent defensive layers, because a restricted `exec()` alone is not a security boundary.

Our contributions are:

- an execution contract (a `RESULT` dictionary plus at least two cross-checking assertions) that turns some silent logic errors into observations the loop can act on;
- a layered sandbox (static AST gate, restricted builtins with a guarded import, a scrubbed child process, CPU and memory limits, a wall-clock timeout) evaluated against 17 attacks;
- a deterministic fault injector that plants crash or logic bugs in the first attempt, together with an offline attribution procedure that separates bugs the checks caught from bugs that slipped through;
- a benchmark of 8 problems with hand-written reference solutions, comparing one-shot generation with the loop under two models; and
- an open-source web application that streams the loop live (https://github.com/DeonMenezes-11/QuantSolve).

II. Related Work

Reasoning and acting. ReAct [1] interleaves free-text reasoning with tool calls and their observations. In QuantSolve one ReAct step is a whole attempt: the Thought is the model's plan, the Action is a complete program and the Observation is the sandbox result.

Learning from feedback. Reflexion [2] keeps verbal reflections from failed trials in an episodic memory. Self-Refine [5] uses the model as its own critic, and Self-Debugging [6] shows that execution results and explanations of code improve repair. Huang *et al.* [7] report that LLMs struggle to self-correct reasoning *without* external feedback. Our loop always grounds

Algorithm 1 QuantSolve self-healing loop

Input: task T, attempt budget K, optional fault mode f
Output: verified `RESULT` and answer, or failure

$M \leftarrow [\,]$ ▷ Reflexion memory
for $k = 1$ **to** K **do**
  (thought, c) $\leftarrow$ SOLVE(T, c_{k-1}, o_{k-1}, M)
  if $k = 1$ **and** $f \neq \varnothing$ **then** $c \leftarrow$ INJECT(c, f)
  $o_k \leftarrow$ SANDBOX(c) ▷ gate, child process, checks
  if o_k.ok **then return** SUMMARISE$(T, o_k.\texttt{RESULT})$
  $M \leftarrow M \cup \{\text{REFLECT}(T, c, o_k, M)\}$
end for
return failure with the last observation

correction in an external signal, either the interpreter or a failed assertion.

Programs as reasoning. PAL [3] and Program of Thoughts [4] delegate computation to an interpreter. Weng *et al.* [8] show that self-verification improves reasoning. We make verification part of the program itself, as executable assertions.

Executing untrusted code. Benchmarks such as HumanEval run model-written code in isolated environments [9]. We target a lighter, laptop-friendly setting and combine static and runtime defences built on the Python standard library [10].

III. System Design

A. The self-healing loop

Fig. 1 and Algorithm 1 summarise the loop. Each attempt asks the model, in JSON mode, for a short plan and a complete program. On a failure, a separate reflection call returns a diagnosis, a general lesson and a plan for the next attempt. All lessons so far, together with the previous program and its observation, are passed to the next attempt. A loop guard hashes the normalised abstract syntax tree (AST) of each program and records an error signature (failure kind, exception type and failing source line). When either repeats, the next prompt carries an explicit "change approach" nudge at a higher sampling temperature. When a program passes, a final call writes a plain-English answer that quotes the verified `RESULT` without recomputing it.

B. Execution contract and self-verification

A program must compute every number from the task, store exactly the requested keys in a JSON-serialisable `RESULT` dictionary, and contain at least two assertions. Each assertion must compare quantities obtained in *different* ways, for example a month-by-month amortisation against the closed-form interest total, or a dynamic-programming count against *total minus complement*. Floats must be compared with a tolerance. Programs with fewer than two assertions are rejected before they run. Every outcome is classified into one of six failure kinds (*blocked*, *syntax*, *runtime*, *assertion*, *timeout*, *contract*), and tracebacks are trimmed to the frames of the generated program, with each source line attached.

C. Layered sandbox

(1) Static AST gate. Imports are limited to an allow-list (`math`, `statistics`, `fractions`, `decimal`, `itertools`, `numpy` and a few others). The names `eval`, `exec`, `open`, `getattr`, `__import__` and similar are rejected, along with every attribute that starts with an underscore and attributes that lead out of the sandbox (frame internals, re-exported modules, NumPy file I/O). *(2) Restricted builtins* with a guarded `__import__` that re-checks the allow-list at run time. *(3) A fresh `multiprocessing` child* ("spawn" start method) that wipes its environment before running user code, so the API key is never present. *(4) Resource limits*: `RLIMIT_CPU` and a 1 GB memory watchdog. *(5) A wall-clock timeout* (10 s by default), after which the child is terminated.

D. Fault injection and attribution

To demonstrate and measure healing, an optional injector rewrites the first program with an `ast.NodeTransformer`. *Crash* mode misspells one use of a variable, which raises a `NameError`. *Logic* mode makes the first `range()` loop stop one step early or flips a +/− inside a loop. Because the injection happens after the model has written its code, an attempt-1 failure does not prove that the checks caught the injected bug. We therefore re-run the model's original program offline and label each case *caught* (the original passes but the mutant fails), *silent* (both pass with different numbers), *benign* (identical numbers), *original failed*, or *not applied*.

E. Web application

A FastAPI server exposes a JSON API and runs each solve in a background thread. The loop's event callback appends to a per-job log, which is streamed to the browser as Server-Sent Events [11] (*attempt start*, *action*, *observation*, *reflection*, *final*). Because the log can be replayed from any position, a page refresh resumes a running job, and a stop flag is checked between steps. The front end is a hand-written single-page application without a framework. It provides a pipeline stepper that shows where each attempt failed, per-attempt tabs (Thought, Code, Observation, Reflection) with the failing and injected lines highlighted and a diff against the previous attempt, a history of saved runs, benchmark dashboards and an explainer (Figs. 2–6). The server binds to the local host only, and model output is always rendered as text, never as HTML.

IV. Experimental Setup

Problems. Table I lists the 8 problems. Each states every rule and the exact `RESULT` keys, and has a hand-written reference solution cross-checked by an independent method in the test suite. A run is *correct* only if every scored key is within its tolerance of the reference.

Arms. *One-shot* allows a single attempt. *Loop* is the full system with $K{=}5$ attempts. *Loop + logic bug* injects a logic fault into attempt 1. *Loop + crash bug* injects a crash fault on 3 problems. Every case was repeated 2 times

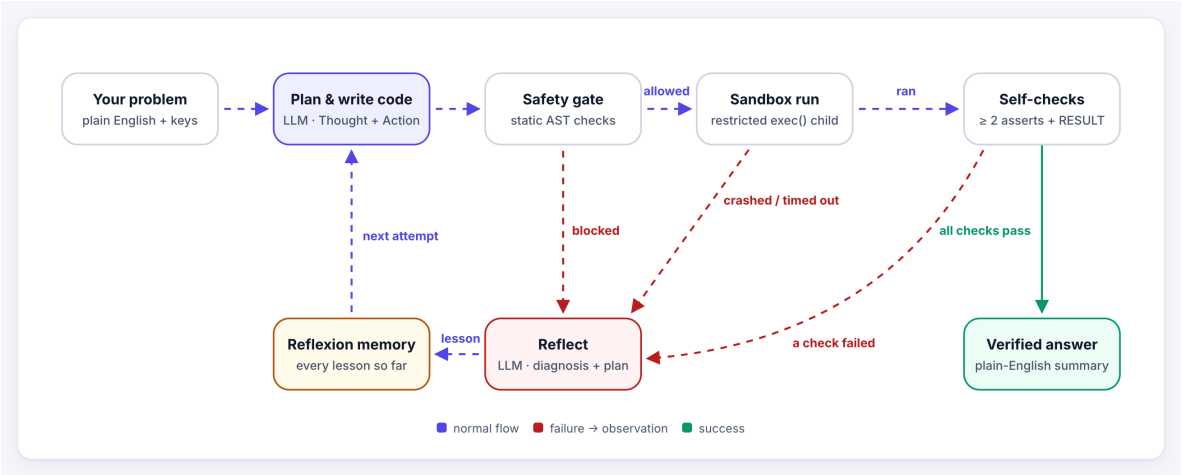


Fig. 1. The QuantSolve loop as drawn in the web application. Blue: normal flow. Red: a failure that becomes the observation for the next attempt. Green: all self-checks passed. A reflection is stored in memory after every failure and shown to every later attempt.

TABLE I
BENCHMARK PROBLEMS AND CORRECT LOOP RUNS

Problem	Domain	Keys	Loop
Birthday problem: pairs and triples (exact)	Probability	4	1/2
Compound interest with changing rates and deposits	Interest	3	2/2
Counting strings with adjacency constraints	Combinatorics	3	0/2
Order quantity with quantity discounts	Operations	4	2/2
Indian income tax, new regime FY 2024-25	Taxation	4	2/2
Home loan with yearly prepayments	Loans	4	2/2
Best launch angle with linear air drag	Physics	3	2/2
Step-up SIP future value	Investing	4	2/2

TABLE II
BENCHMARK SUMMARY (GPT-4.1-MINI, K=5)

Arm	Runs	Correct	Verif.	V. wrong	Att.	Healed	¢/run
One-shot	16	11 (69%)	12	1	1.00	0	0.24
Loop	16	13 (81%)	16	3	1.19	2	0.29
Loop + logic bug	16	8 (50%)	16	8	1.88	7	0.49
Loop + crash bug	6	5 (83%)	5	0	3.00	5	0.81

TABLE III
ATTRIBUTION OF ATTEMPT-1 OUTCOMES IN THE INJECTED ARMS

Arm	Caught	Silent	Benign	Orig. failed	N/A
Loop + logic bug	2	5	2	5	2
Loop + crash bug	4	0	0	2	0

with `gpt-4.1-mini` [12], giving 54 runs in total for an estimated US$0.21. The same benchmark was first run with `gpt-4o-mini`.

Metrics. *Accuracy* against the reference, *verified* (the final program passed its own checks), *verified but wrong*, attempts per run, *healed* (attempt 1 failed and a later attempt passed), and cost estimated from token counts.

V. RESULTS

A. Does the loop help?

Table II and Fig. 4 give the main results. With `gpt-4.1-mini`, the loop raised accuracy from 69% to 81% (+12 points) and used only 1.19 attempts per run on average, because most first programs already passed. A loop run cost 0.29 US cents, compared with 0.24 for one-shot. In the one-shot arm most failures were failed assertions, which is exactly the signal the loop exploits. On the other hand, 3 loop runs ended *verified but wrong*: they passed their own checks yet disagreed with the reference. The weakest problems were *Counting strings with adjacency constraints* and *Birthday problem: pairs and triples (exact)*, where the checks tended to agree with an incorrect method.

B. Injected bugs

Crash faults are easy to repair. The cleaned traceback names the line and often suggests the correct identifier, and 5 of 6 crash runs recovered (83% correct, 3.00 attempts). Logic faults expose the limits of self-written checks (Table III). Of the 9 logic faults that could be attributed, the checks caught 2, 5 changed the answer while every check still passed, and 2 did not change the answer. In the remaining runs the model's own program had already failed (5) or the injector found nothing to mutate (2). The logic-bug arm therefore ends with 8 verified-but-wrong runs and 50% accuracy.

C. Effect of the base model

Table IV compares the two models on the same benchmark. With `gpt-4o-mini` the loop moved accuracy from 6% to 31% (+25 points), but 7 of its loop runs were verified but wrong. Raw point gains favour the weaker model only because

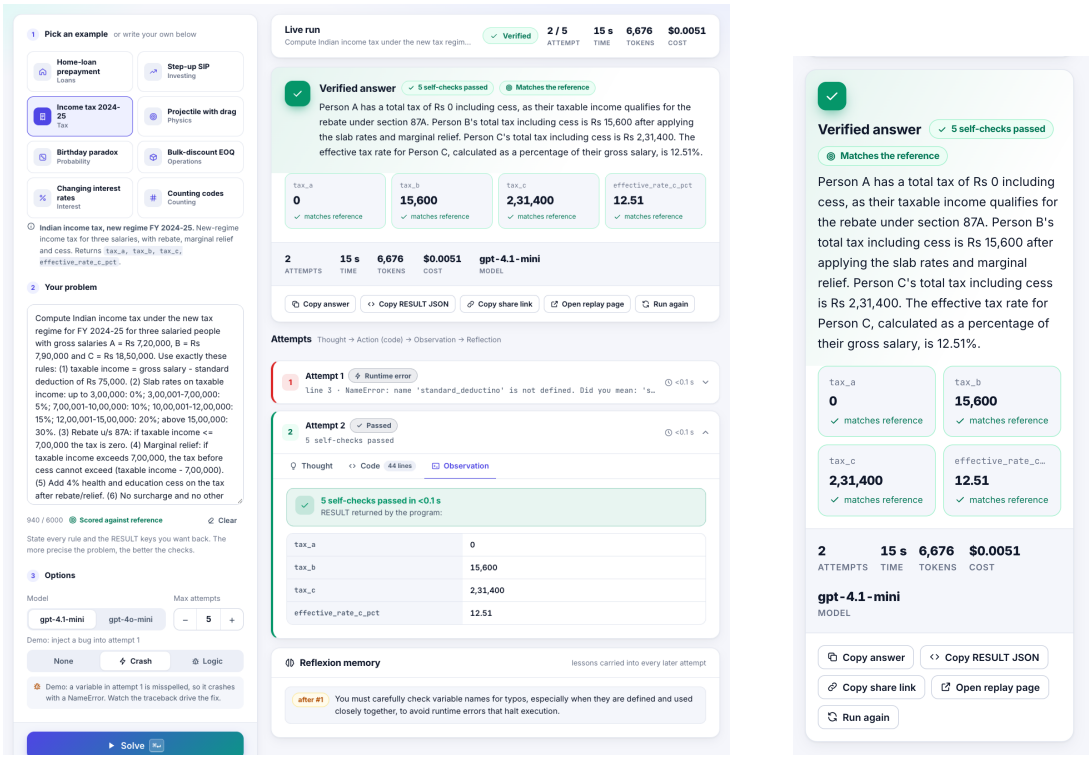


Fig. 2. Left: the Solve page after a live run (example gallery and options on the left; run header, verified answer checked against the reference, attempt timeline and Reflexion memory on the right). Right: the same answer on a 390-pixel-wide phone screen.

TABLE IV
CORRECT RUNS PER ARM FOR BOTH MODELS

Model	One-shot	Loop	+Logic	+Crash	Errors fixed
`gpt-4o-mini`	1/16	5/16	4/16	2/6	27%
`gpt-4.1-mini`	11/16	13/16	8/16	5/6	40%

it starts lower. Measured as the share of one-shot errors that the loop fixed, the gain was 27% for `gpt-4o-mini` and 40% for `gpt-4.1-mini`. Self-correction adds no knowledge. It turns a failed check into another attempt, and that attempt helps only if the model can read the observation, find the cause and write a correct fix.

D. Per-problem behaviour and safety

Fig. 5 breaks the results down by problem. Finance, tax, physics and inventory problems were solved reliably, while exact combinatorics remained hard in every arm. In the safety evaluation all 17 of 17 attack programs were contained: 11 were rejected by the AST gate, 2 were stopped by the timeout and 1 by the memory watchdog. The remaining 3 were run with the gate disabled on purpose. These showed that the restricted builtins, the guarded import and the environment scrub still hold on their own. In one of them a generator frame walk reached the real `os.environ` of the child process, but found no key there.

VI. The Web Application

Fig. 2 shows the Solve page after a live run on the income-tax problem with the crash fault enabled. Attempt 1 failed with a `NameError` on the injected line, the reflection was stored in memory, and attempt 2 passed all of its checks. Every value is marked against the reference answer. Fig. 3 shows the results dashboard and the run history, where coloured squares trace each run's path through its attempts, and Fig. 6 shows an attempt card with the injected fault and the failing assertion highlighted. The interface supports light and dark themes and adapts to phone screens.

VII. Discussion and Limitations

Self-written checks share the model's blind spots. Assertions turned some silent errors into observable failures, but most injected logic bugs that changed the answer still passed the checks. Checks can also be wrong themselves, which wastes attempts. Generating the checks in a separate call from the problem statement alone, using property-based tests, or having a second model review the checks would make verification more independent.

The sandbox is defence in depth, not a hard boundary. Restricted `exec()` in CPython can be escaped by introspection, which the AST gate blocks by pattern. Any unknown escape would then meet only the process boundary and the scrubbed environment. For untrusted users a container, an OS sandbox profile or a WebAssembly build of Python would be the right base.

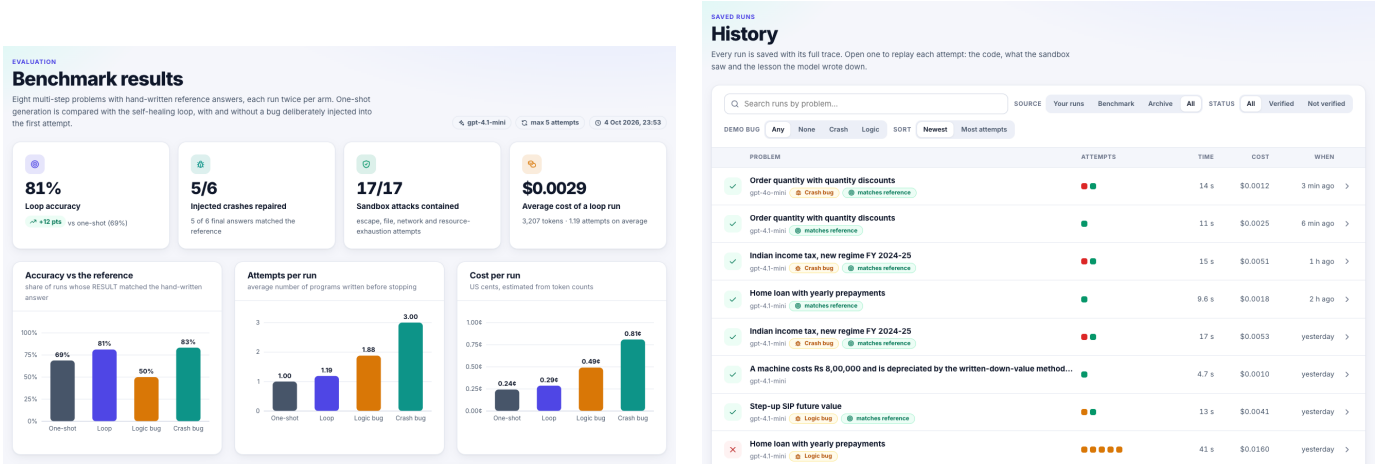


Fig. 3. Left: the results dashboard, which reads the benchmark and safety files. Right: the run history with search and filters; each square is one attempt (red: crash, amber: failed self-check, green: passed).

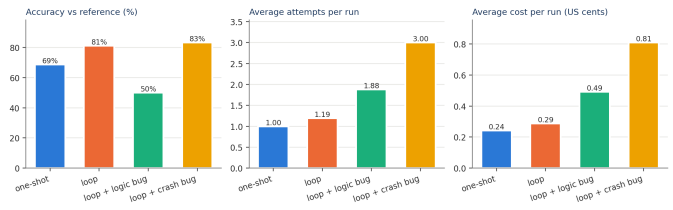


Fig. 4. Accuracy, average attempts and average cost per run for each arm.

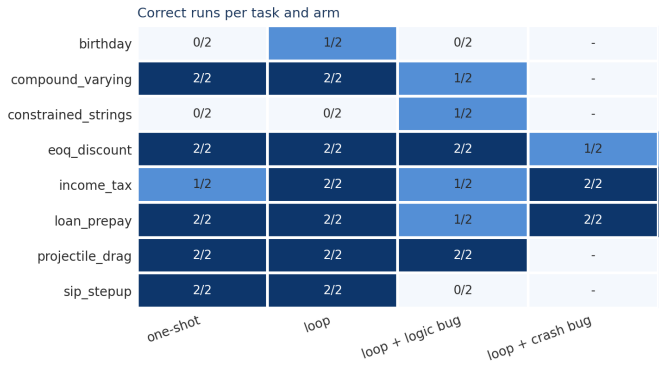


Fig. 5. Correct runs out of all runs for each problem and arm.

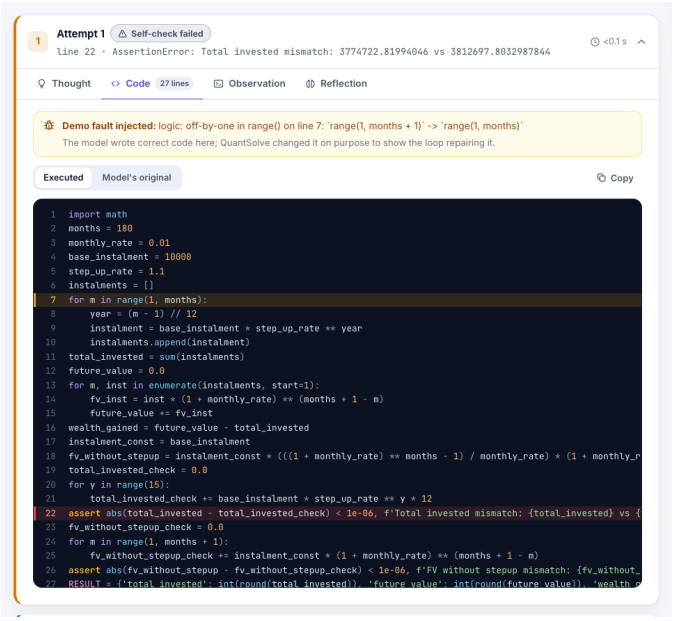


Fig. 6. An attempt card (Code tab) from a replayed run. The injected off-by-one is explained in a banner, the mutated line is marked in amber and the failing assertion in red.

Small benchmark. With 8 problems and 2 repeats per case, differences of one or two runs should not be over-read. Larger models were not tested, to keep the cost low.

VIII. Conclusion

QuantSolve implements a complete self-correction loop for quantitative problems: program-aided reasoning, an execution contract with self-verifying assertions, a layered sandbox, Reflexion memory and a live web interface. The loop improved accuracy at roughly the cost of one extra model call per failure, and it repaired injected crashes reliably. It is only as good as the checks, though. It can repair crashes and failed checks, but it cannot repair an error that no check looks at.

The code, results, traces and web application are available at https://github.com/DeonMenezes-11/QuantSolve.

Code Availability

The source code, benchmark traces, results and the web application are openly available at https://github.com/DeonMenezes-11/QuantSolve. The command `python3 web.py` starts the website, and `python3 -m pytest` runs the offline test suite.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
[3] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023.
[4] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Trans. Mach. Learn. Res.*, 2023.
[5] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
[6] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
[7] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
[8] Y. Weng, M. Zhu, F. Xia, B. Li, S. He, S. Liu, B. Sun, K. Liu, and J. Zhao, "Large language models are better reasoners with self-verification," in *Findings Assoc. Comput. Linguist.: EMNLP*, 2023.
[9] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
[10] Python Software Foundation, "The Python standard library: `ast`, `multiprocessing` and `resource`," Python 3.13 documentation. [Online]. Available: https://docs.python.org/3/library/
[11] WHATWG, "Server-sent events," *HTML Living Standard*. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html
[12] OpenAI, "Models," OpenAI API documentation. [Online]. Available: https://platform.openai.com/docs/models